

循序渐进，学习开发 xv6-riscv

第 11 章 自旋锁

汪辰



目录

01

并发与同步

02

临界区与锁

03

自旋锁的设计和实现

04

自旋锁的应用



01

并发与同步

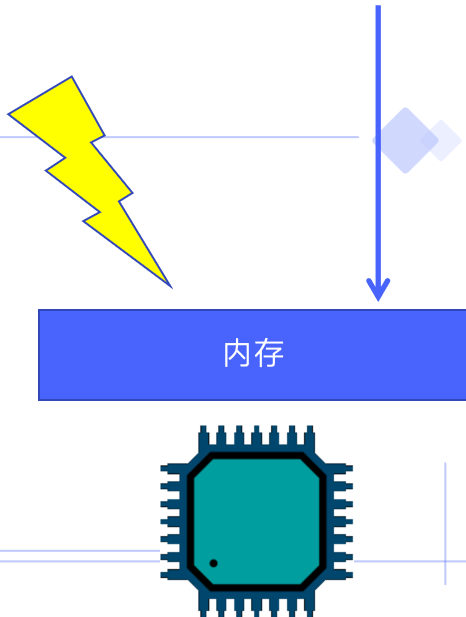
并发与同步

- **并发 (Concurrency)** 指多个控制流同时执行，包括以下几种情况：
 - ✓ 任务和中断的并发（无论单个 Hart 还是多个 Hart 都存在）
 - ✓ 单个 Hart 上多个任务的并发（交替执行）
 - ✓ 多个 Hart 上多任务的并发（真正的并发）

并发与同步

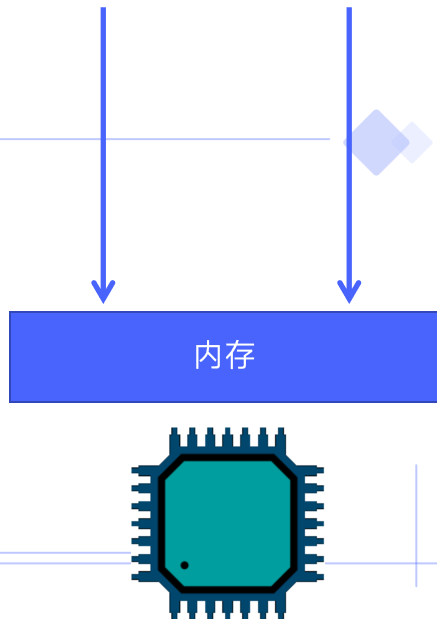
• **并发 (Concurrency)** 指多个控制流同时执行，包括以下几种情况：

- ✓ 任务和中断的并发（无论单个 Hart 还是多个 Hart 都存在）
- ✓ 单个 Hart 上多个任务的并发（交替执行）
- ✓ 多个 Hart 上多任务的并发（真正的并发）



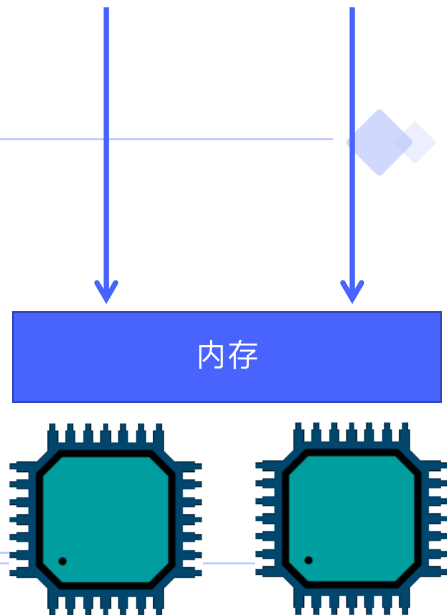
并发与同步

- **并发 (Concurrency)** 指多个控制流同时执行，包括以下几种情况：
 - ✓ 任务和中断的并发（无论单个 Hart 还是多个 Hart 都存在）
 - ✓ 单个 Hart 上多个任务的并发（交替执行）
 - ✓ 多个 Hart 上多任务的并发（真正的并发）



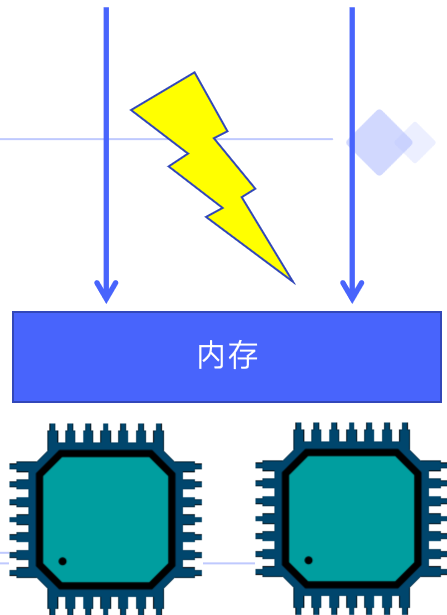
并发与同步

- **并发 (Concurrency)** 指多个控制流同时执行，包括以下几种情况：
 - ✓ 任务和中断的并发（无论单个 Hart 还是多个 Hart 都存在）
 - ✓ 单个 Hart 上多个任务的并发（交替执行）
 - ✓ 多个 Hart 上多任务的并发（真正的并发）



并发与同步

- **并发 (Concurrency)** 指多个控制流同时执行，包括以下几种情况：
 - ✓ 任务和中断的并发（无论单个 Hart 还是多个 Hart 都存在）
 - ✓ 单个 Hart 上多个任务的并发（交替执行）
 - ✓ 多个 Hart 上多任务的并发（真正的并发）
- **同步** 是为了保证在并发执行的环境中各个控制流可以有效执行而采用的一种编程技术，也称为 **并发控制**。





02

临界区与锁



临界区

临界区 (Critical Section)：在并发的程序执行环境中，一个会访问共享资源（例如：一个共享设备或者一块共享存储内存）的**指令片段**，而且当这样的多个指令片段同时访问某个共享资源时可能会引发问题。

临界区

```
int balance = 200;
```

读取余额 balance

判断余额是否 ≥ 100

如果条件满足则余额减去
100 元

⋮

instruction #M



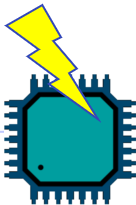
读取余额 balance

判断余额是否 ≥ 200

如果条件满足则余额减去
200 元

⋮

instruction #N



临界区

```
int balance = 200;
```

读取余额 balance

判断余额是否 ≥ 100

如果条件满足则余额减去
100 元

⋮

instruction #M

临界区

读取余额 balance

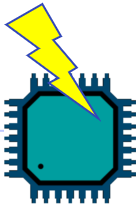
判断余额是否 ≥ 200

如果条件满足则余额减去
200 元

⋮

instruction #N

临界区



实现同步的技术 - 锁

- 在并发环境下为了有效控制临界区的执行（同步），我们要做的是当有一个控制流进入临界区时，其他相关控制流必须 **等待**。
- 锁是一种最常见的用来实现同步的技术。提供互斥功能，确保同一时间只有一个 CPU 可以持有锁。

锁如何实现同步



共享资源

acquire (lock)

instruction i

instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)

acquire (lock)

instruction i

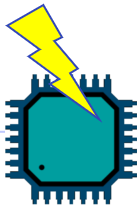
instruction i + 1

instruction i + 2

⋮

instruction i + m

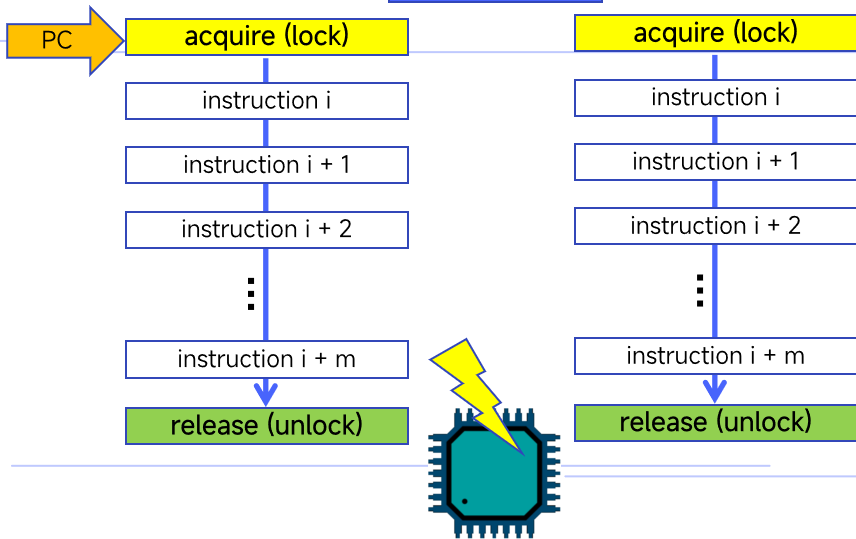
release (unlock)



锁如何实现同步



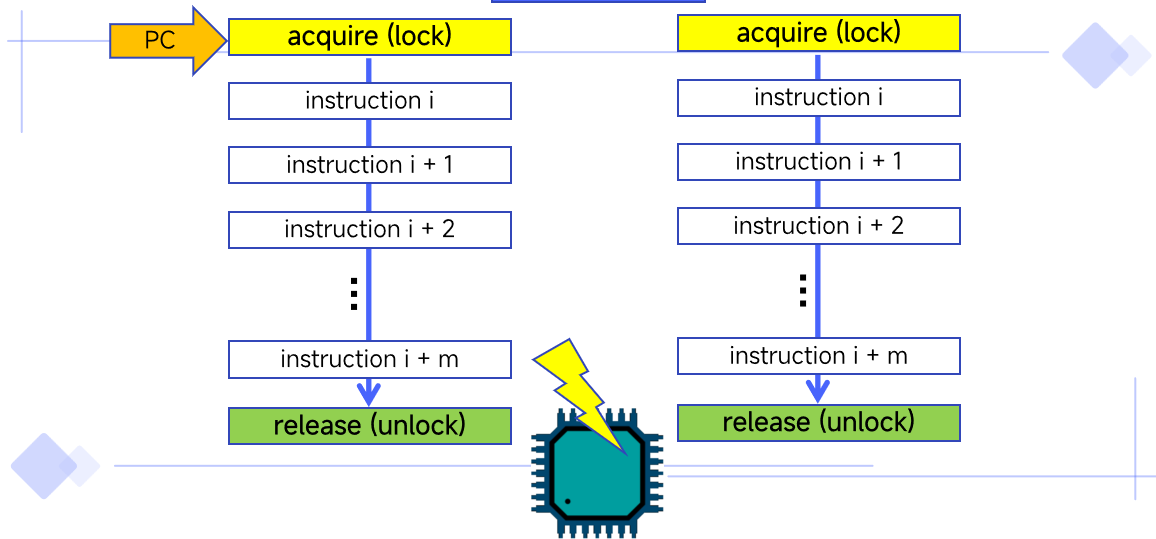
共享资源



锁如何实现同步



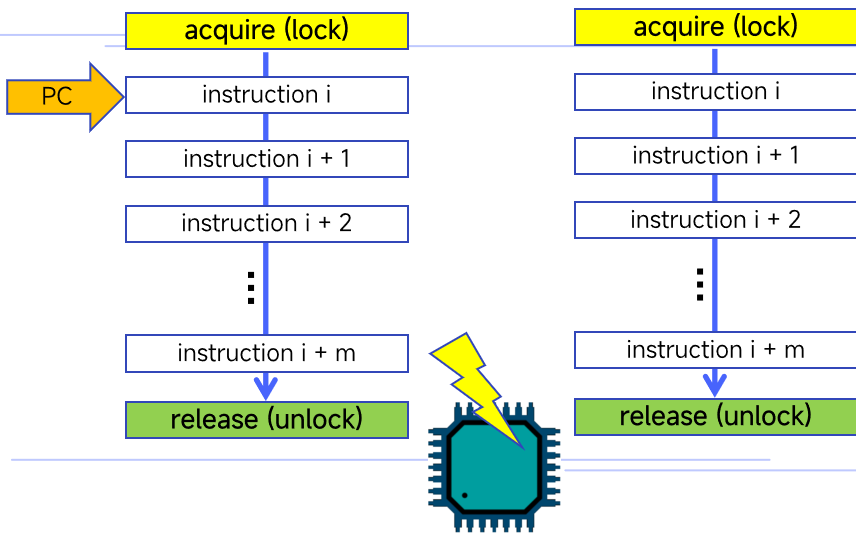
共享资源



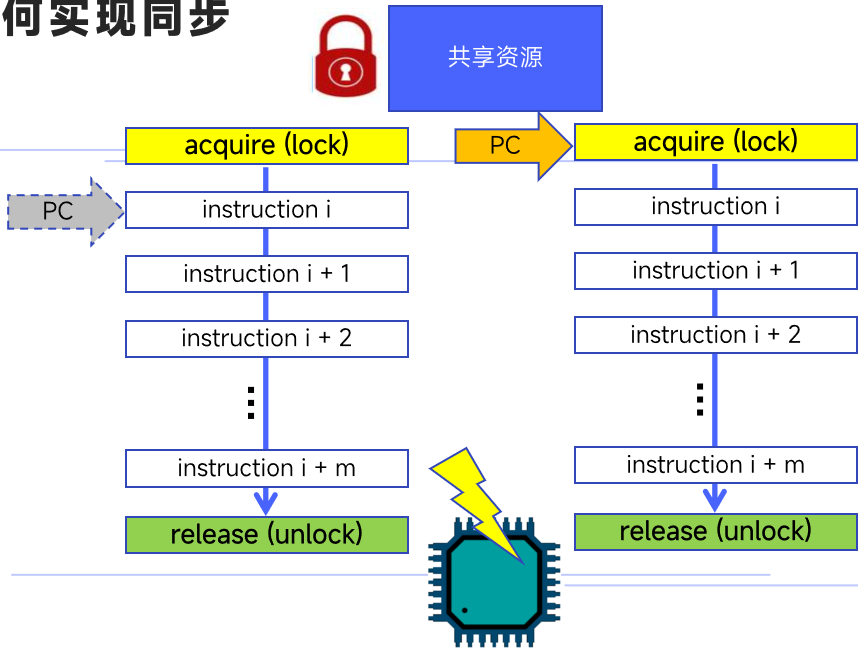
锁如何实现同步



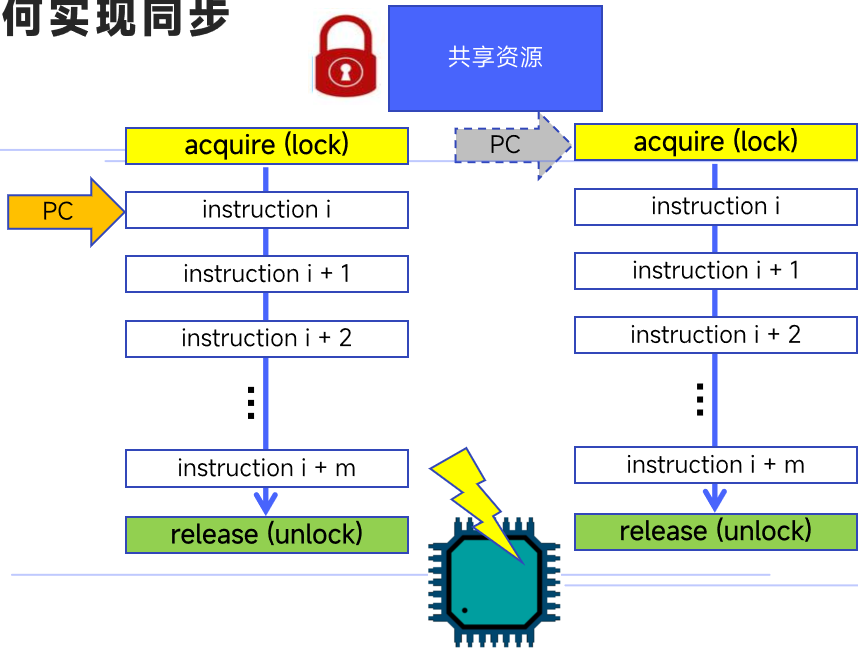
共享资源



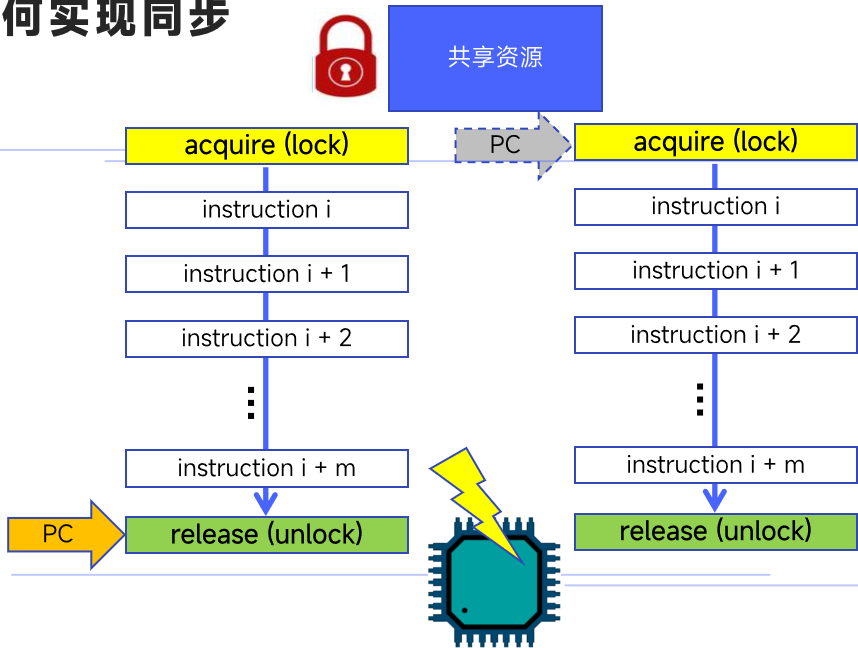
锁如何实现同步



锁如何实现同步



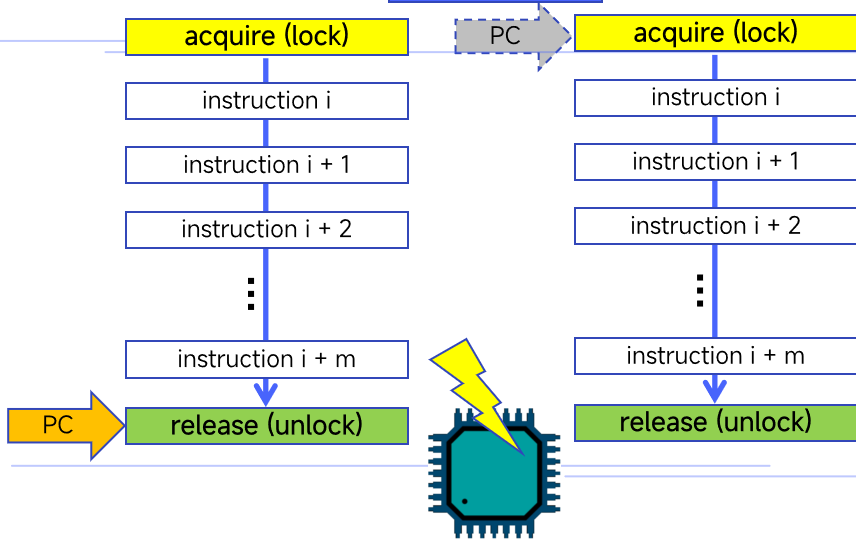
锁如何实现同步



锁如何实现同步



共享资源



锁如何实现同步



共享资源



acquire (lock)

instruction i

instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)

PC



acquire (lock)

instruction i

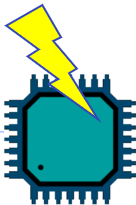
instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)



锁如何实现同步



共享资源

acquire (lock)

instruction i

instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)

PC

acquire (lock)

instruction i

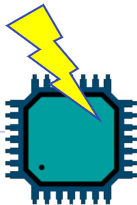
instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)



锁如何实现同步



共享资源

acquire (lock)

instruction i

instruction i + 1

instruction i + 2

⋮

instruction i + m

release (unlock)

PC

acquire (lock)

instruction i

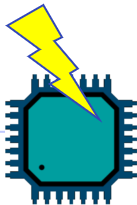
instruction i + 1

instruction i + 2

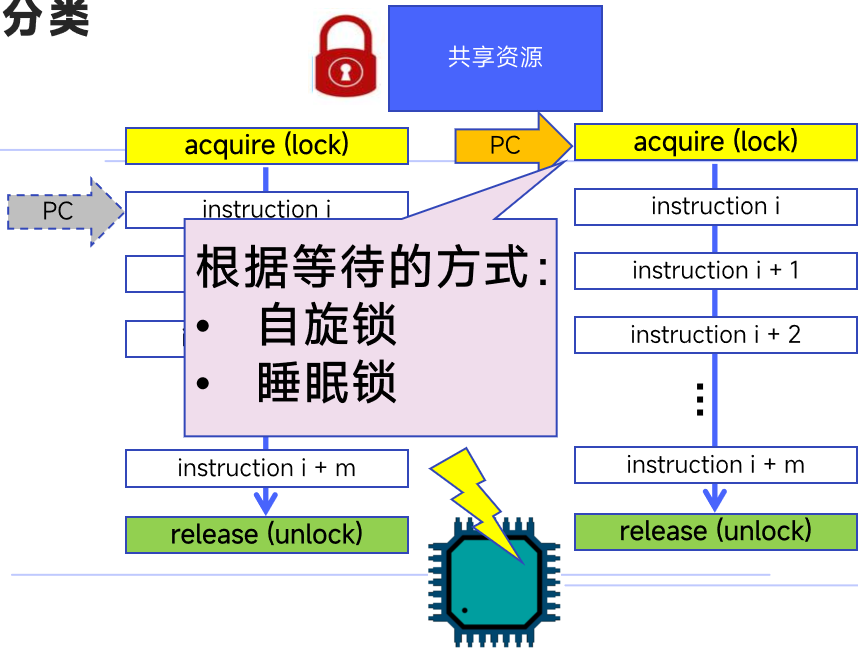
⋮

instruction i + m

release (unlock)



锁的分类



其他同步技术

同步技术	描述
自旋锁 (Spin Lock)	如果一个任务试图获得一个已经被持有的自旋锁，这个任务就会进入忙循环 (busy loops, 即自旋) 并等待，直到该锁可用，否则该任务就可以立刻获得这个锁并继续执行。自旋锁可以防止多个任务同时进入临界区 (critical region)
信号量 (Semaphore)	信号量是一种睡眠锁，当任务请求的信号量无法获得时，就会让任务进入等待队列并且让任务睡眠。当信号量可以获得时，等待队列中的一个任务就会被唤醒并获得信号量。
互斥锁 (Mutex)	互斥锁可以看作是对互斥信号量 (count 为 1) 的改进，是一种特殊的信号量处理机制
完成变量 (Completion Variable)	一个任务执行某些工作时，另一个任务就在完全变量上等待，当前者完成工作，就会利用完全变量来唤醒所有在这个完全变量上等待的任务。
.....	

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc starts from the top of the outer circle and curves around to the bottom. In the top right corner, there is a solid blue rectangle.

03

自旋锁的设计和实现

一个现实的竞争案例

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    r->next = kmem.freelist;
    kmem.freelist = r;
}
```

一个现实的竞争案例

```
void kfree(void *pa)
{
    struct run *r;

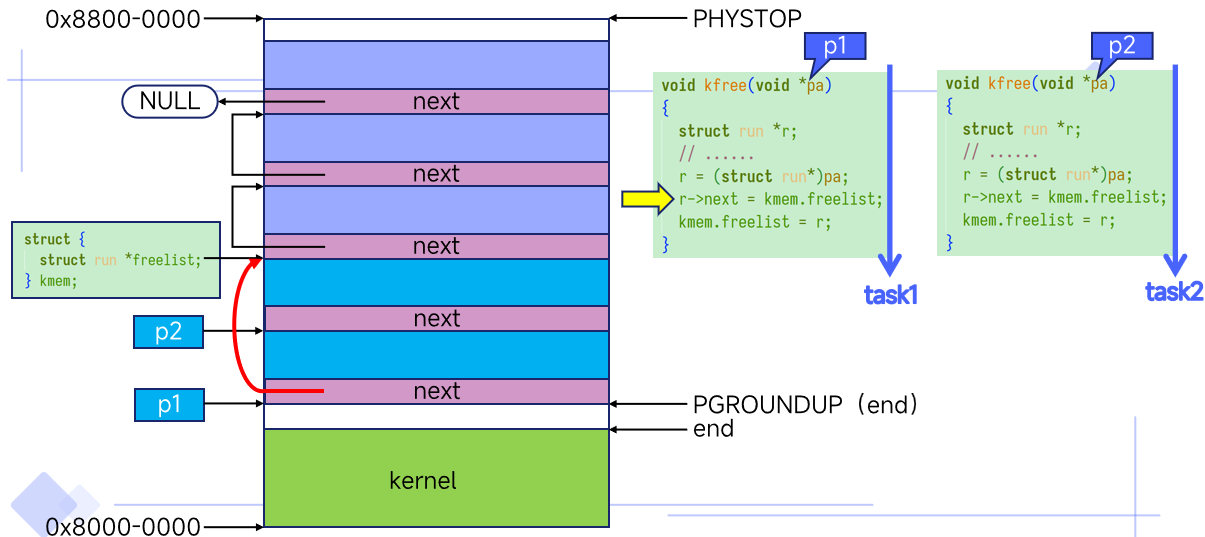
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

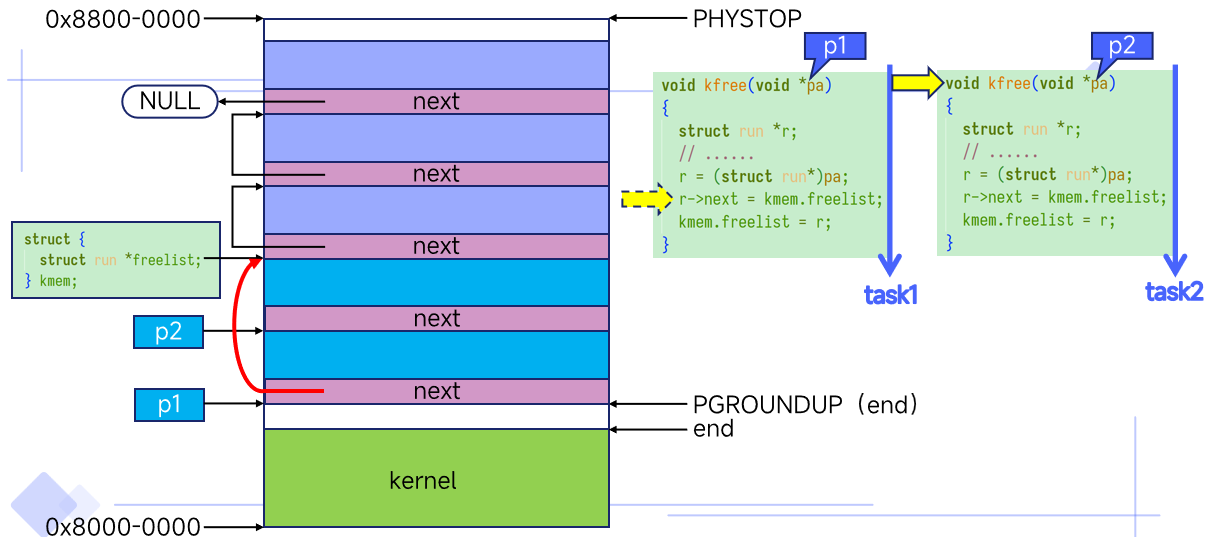
    r = (struct run*)pa;

    r->next = kmem.freelist;
    kmem.freelist = r;
}
```

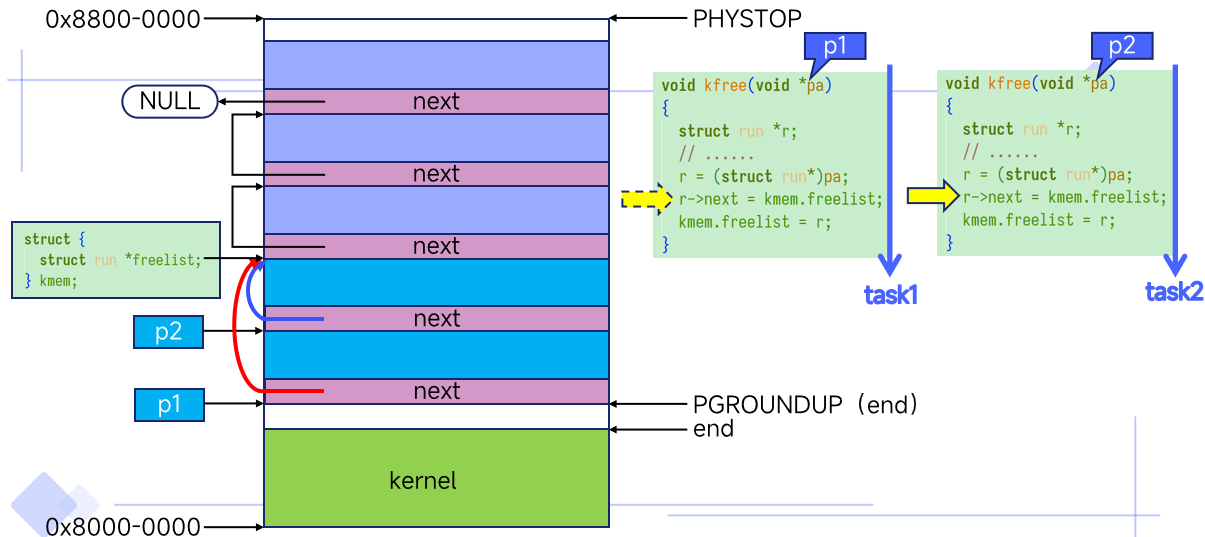

一个现实的竞争案例



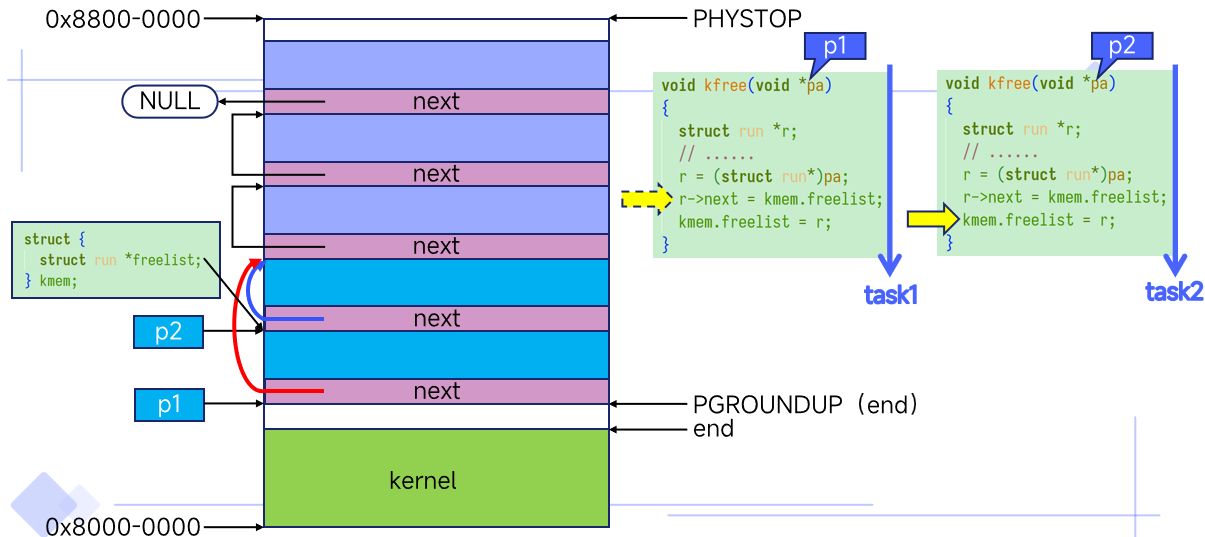
一个现实的竞争案例



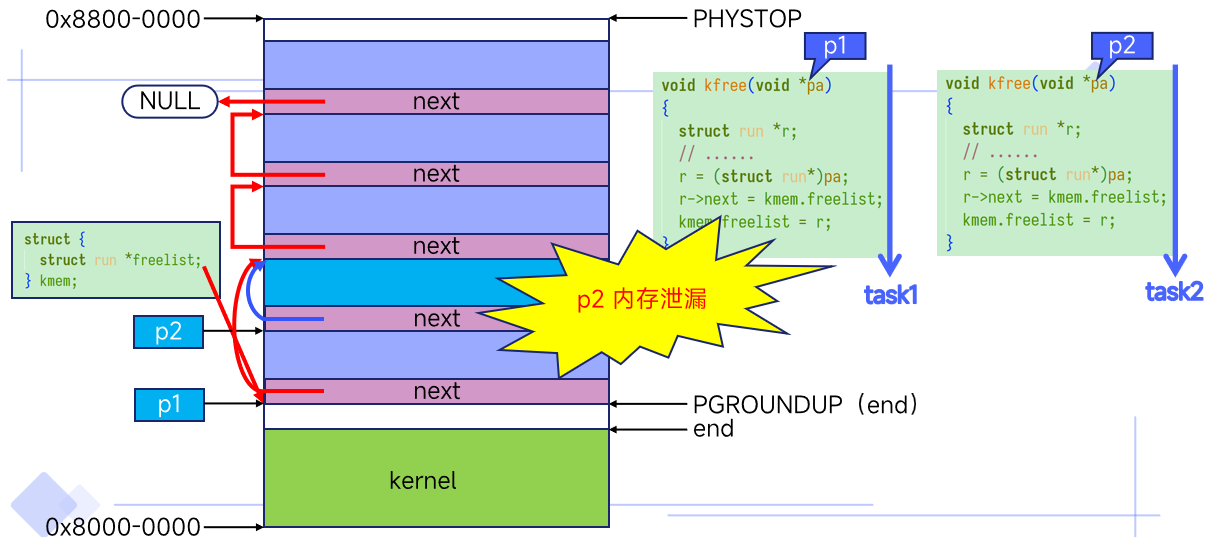
一个现实的竞争案例



一个现实的竞争案例



一个现实的竞争案例



自旋锁的设计与实现 - API

```
// Mutual exclusion lock.  
struct spinlock {  
    uint locked;    // Is the lock held?  
    // .....  
};
```

```
void    acquire(struct spinlock*);  
void    release(struct spinlock*);
```

自旋锁的设计与实现 - API

```
// Mutual exclusion lock.  
struct spinlock {  
    uint locked;    // Is the lock held?  
    // .....  
};
```

```
void    acquire(struct spinlock*);  
void    release(struct spinlock*);
```

自旋锁的使用:

```
struct spinlock lock;  
  
// .....  
acquire(&lock);  
// critical section  
release(&lock);  
// .....
```

自旋锁的设计与实现 - 原子性

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

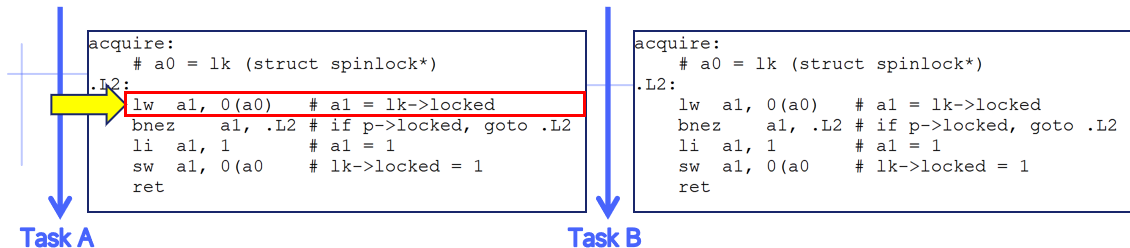

自旋锁的设计与实现 - 原子性

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

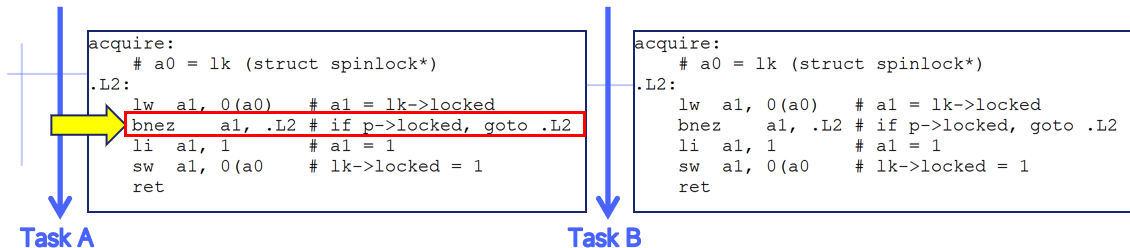


```
acquire:
    # a0 = lk (struct spinlock*)
.L2:
    lw  a1, 0(a0)    # a1 = lk->locked
    bnez a1, .L2     # if p->locked, goto .L2
    li  a1, 1        # a1 = 1
    sw  a1, 0(a0)    # lk->locked = 1
    ret
```

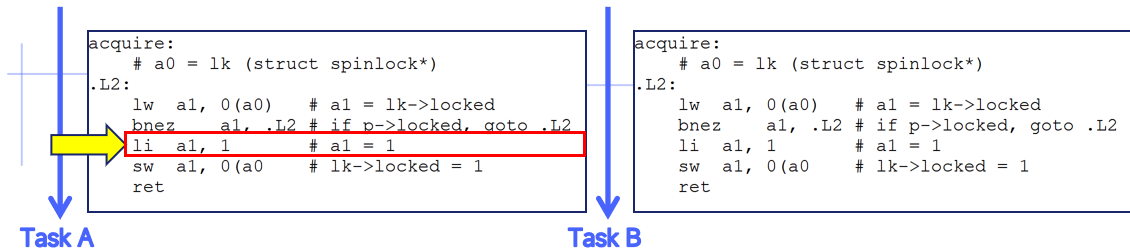
自旋锁的设计与实现 - 原子性



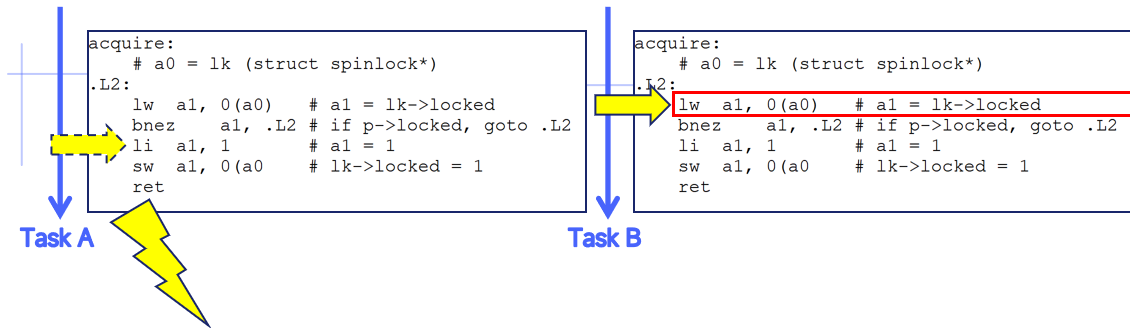
自旋锁的设计与实现 - 原子性



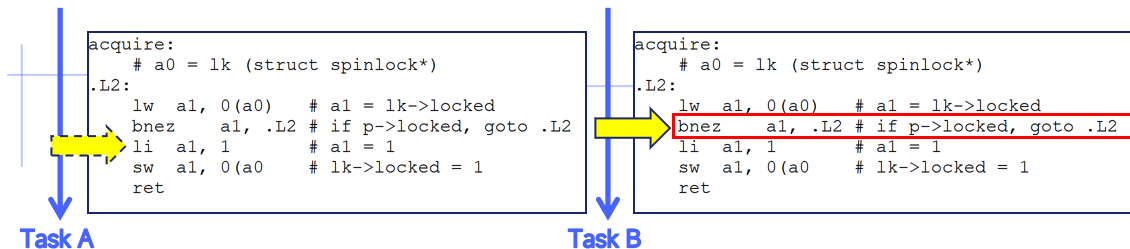
自旋锁的设计与实现 - 原子性



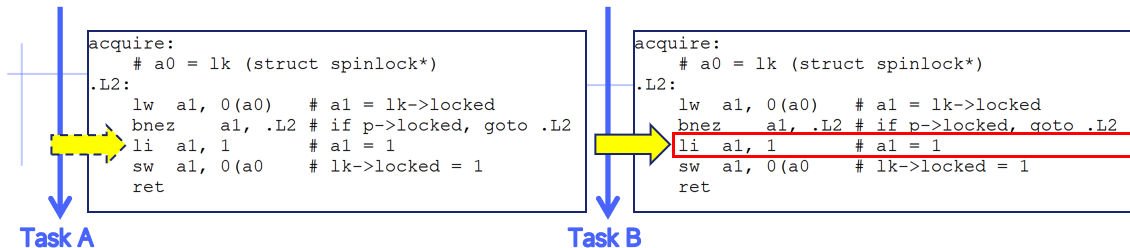
自旋锁的设计与实现 - 原子性



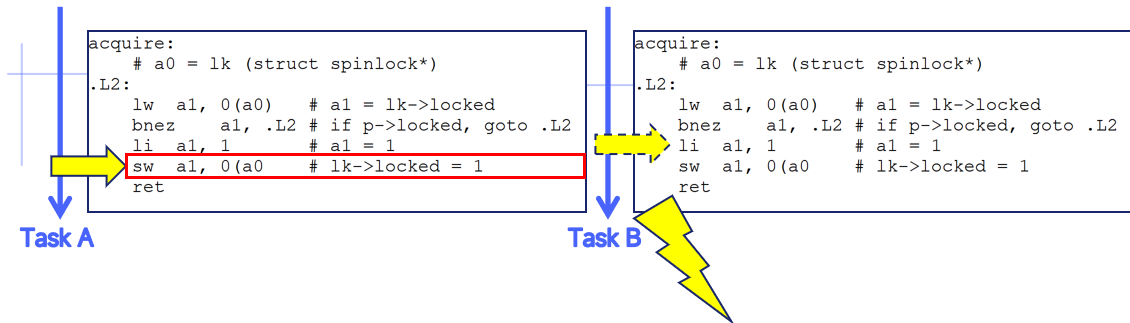
自旋锁的设计与实现 - 原子性



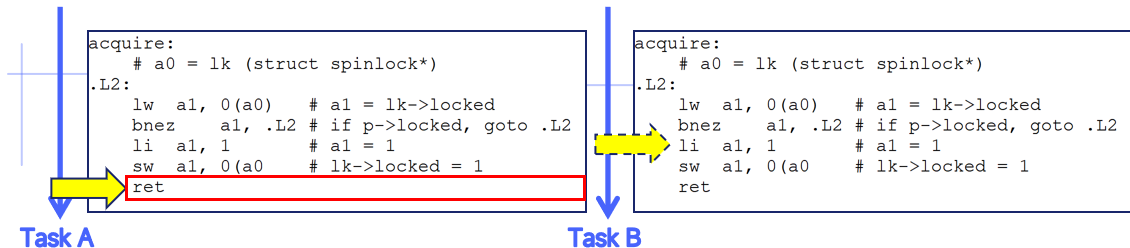
自旋锁的设计与实现 - 原子性



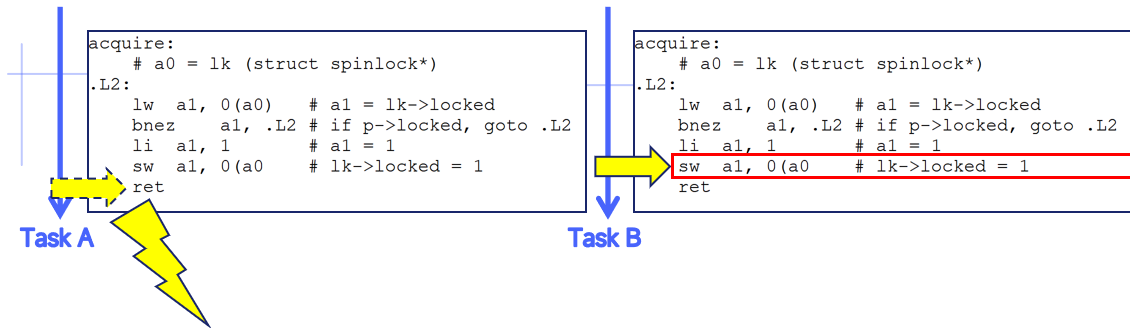
自旋锁的设计与实现 - 原子性



自旋锁的设计与实现 - 原子性



自旋锁的设计与实现 - 原子性



自旋锁的设计与实现 - 原子性

```
acquire:
    # a0 = lk (struct spinlock*)
.L2:
    lw  a1, 0(a0)    # a1 = lk->locked
    bnez a1, .L2     # if p->locked, goto .L2
    li  a1, 1        # a1 = 1
    sw  a1, 0(a0)    # lk->locked = 1
    ret
```

Task A

```
acquire:
    # a0 = lk (struct spinlock*)
.L2:
    lw  a1, 0(a0)    # a1 = lk->locked
    bnez a1, .L2     # if p->locked, goto .L2
    li  a1, 1        # a1 = 1
    sw  a1, 0(a0)    # lk->locked = 1
    ret
```

Task B

自旋锁的设计与实现 - 原子性

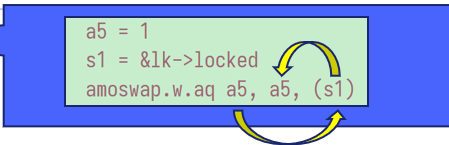
```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

以 *原子 (atomic)* (不可分割的) 的方式执行判断和上锁。

自旋锁的设计与实现 - 原子性

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

a5 = 1
s1 = &lk->locked
amoswap.w.aq a5, a5, (s1)



13.4. "Zaamo" Extension for Atomic Memory Operations

31	27	26	25	24	20	19	15	14	12	11	7	6	0							
funct5				aq	rl	rs2			rs1			funct3		rd		opcode				
5					1	1	5			5			3		5		7			
AMOSWAP.W/D					src			addr			width			dest		AMO				
AMOADD.W/D					src			addr			width			dest		AMO				
AMOAND.W/D					src			addr			width			dest		AMO				
AMOOR.W/D					src			addr			width			dest		AMO				
AMOXOR.W/D					src			addr			width			dest		AMO				
AMOMAX[U].W/D					src			addr			width			dest		AMO				
AMOMIN[U].W/D					src			addr			width			dest		AMO				

自旋锁的设计与实现 - 原子性

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}
```

```
a5 = 1
s1 = &lk->locked
amoswap.w.aq a5, a5, (s1)
```

13.4. "Zaamo" Extension for Atomic Memory Operations

31					27	26	25	24				20				19	15				14	12				11	7				6	0			
funct5					aq		rl		rs2				rs1				funct3				rd				opcode										
5					1		1		5				5				3				5				7										
AMOSWAP.W/D									src				addr				width				dest				AMO										
AMOADD.W/D									src				addr				width				dest				AMO										
AMOAND.W/D									src				addr				width				dest				AMO										
AMOOR.W/D									src				addr				width				dest				AMO										
AMOXOR.W/D									src				addr				width				dest				AMO										
AMOMAX[U].W/D									src				addr				width				dest				AMO										
AMOMIN[U].W/D									src				addr				width				dest				AMO										

自旋锁的设计与实现 - 原子性

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```

13.4. "Zaamo" Extension for Atomic Memory Operations

31	27	26	25	24	20	19	15	14	12	11	7	6	0							
funct5				aq	rl	rs2			rs1			funct3		rd		opcode				
5					1	1	5			5			3		5		7			
AMOSWAP.W/D					src			addr			width			dest		AMO				
AMOADD.W/D					src			addr			width			dest		AMO				
AMOAND.W/D					src			addr			width			dest		AMO				
AMOOR.W/D					src			addr			width			dest		AMO				
AMOXOR.W/D					src			addr			width			dest		AMO				
AMOMAX[U].W/D					src			addr			width			dest		AMO				
AMOMIN[U].W/D					src			addr			width			dest		AMO				

自旋锁的设计与实现 - 中断处理

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```


自旋锁的设计与实现 - 中断处理

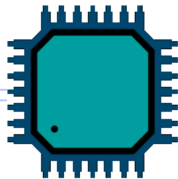
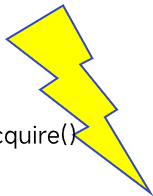
```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```

acquire()



acquire()

自旋锁的设计与实现 - 中断处理

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```



```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire")
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    lk->cpu = mycpu();
}

void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_lock_release(&lk->locked);
    pop_off();
}
```

自旋锁的设计与实现 - 中断处理

```
acquire( A ) { push_off(); ... }
```

临界区 A

```
    acquire( B ) { push_off(); ... }
```

临界区 B

```
    release ( B ) { ... pop_off(); }
```

临界区 A

```
release ( A ) { ... pop_off(); }
```

自旋锁的设计与实现 - 中断处理

```
acquire( A ) { push_off(); ... }
```

临界区 A

```
    acquire( B ) { push_off(); ... }
```

临界区 B

```
    release ( B ) { ... pop_off(); }
```

临界区 A

```
release ( A ) { ... pop_off(); }
```

```
struct cpu {  
    // .....  
    int noff;  
    int intena;  
};
```

```
void push_off(void)  
{  
    int old = intr_get();  
    intr_off();  
    if(mycpu()->noff == 0)  
        mycpu()->intena = old;  
    mycpu()->noff += 1;  
}
```

```
void pop_off(void)  
{  
    struct cpu *c = mycpu();  
    if(intr_get())  
        panic("pop_off - interruptible");  
    if(c->noff < 1)  
        panic("pop_off");  
    c->noff -= 1;  
    if(c->noff == 0 && c->intena)  
        intr_on();  
}
```

自旋锁的设计与实现 - 指令重排

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```



```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```



```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire")
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    lk->cpu = mycpu();
}

void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_lock_release(&lk->locked);
    pop_off();
}
```

自旋锁的设计与实现 - 指令重排

指令重排：为了优化执行效率，编译器和 CPU 可能会改变指令的执行顺序，但保证在单线程下结果与原始顺序一致。

- 编译器重排序
- 指令集并行（CPU 乱序执行）重排序

```
struct spinlock lock;  
void func () {  
    // .....  
    acquire(&lock);  
    // critical section  
    release(&lock);  
    // .....  
}
```

自旋锁的设计与实现 - 指令重排

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```

```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire");
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    __sync_synchronize();
    lk->cpu = mycpu();
}
```

```
void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_synchronize();
    __sync_lock_release(&lk->locked);
    pop_off();
}
```

```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire");
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    lk->cpu = mycpu();
}
```

```
void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_lock_release(&lk->locked);
    pop_off();
}
```

自旋锁的设计与实现 - 完整的改进流程

```
void acquire(struct spinlock *lk)
{
    for(;;) {
        if(lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}
```

```
void acquire(struct spinlock *lk)
{
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
}

void release(struct spinlock *lk)
{
    __sync_lock_release(&lk->locked);
}
```

```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire");
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    __sync_synchronize();
    lk->cpu = mycpu();
}
```

```
void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_synchronize();
    __sync_lock_release(&lk->locked);
    pop_off();
}
```

```
void acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire");
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;
    lk->cpu = mycpu();
}
```

```
void release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");
    lk->cpu = 0;
    __sync_lock_release(&lk->locked);
    pop_off();
}
```




04

自旋锁的应用

第一个例子：

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    r->next = kmem.freelist;
    kmem.freelist = r;
}
```

第一个例子：

```
struct {  
    struct spinlock lock;  
    struct run *freelist;  
} kmem;
```

```
void  
kinit()  
{  
    initlock(&kmem.lock, "kmem");  
    freerange(end, (void*)PHYSTOP);  
}
```

```
// start() jumps here in supervisor mode on all CPUs.  
void main()  
{  
    // .....  
    kinit();           // physical page allocator  
    // .....  
}
```

第一个例子：

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    r->next = kmem.freelist;
    kmem.freelist = r;
}
```

第一个例子：

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    r->next = kmem.freelist;
    kmem.freelist = r;
}
```

临界区

第一个例子：

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

第一个例子:

```
void kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0)
        panic("kfree");

    // Fill with junk to catch da
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

```
void * kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r)
        kmem.freelist = r->next;
    release(&kmem.lock);

    if(r)
        memset((char*)r, 5, PGSIZE); // fill with junk
    return (void*)r;
}
```

第二个例子：

```
int printf(char *fmt, ...)
{
    va_list ap;
    int i, cx, c0, c1, c2;
    char *s;

    va_start(ap, fmt);
    for(i = 0; (cx = fmt[i] & 0xff) != 0; i++){
        // .....
    }
    va_end(ap);

    return 0;
}
```


[illegible]

kernel/trap.c

3

[illegible][illegible]

1

}

}

```
00000000000000000000"
```

```
00000000000000000000>");
```

第二个例子：

```
int printf(char *fmt, ...)
{
    va_list ap;
    int i, cx, c0, c1, c2;
    char *s;
    acquire(&pr.lock);
    va_start(ap, fmt);
    for(i = 0; (cx = fmt[i] & 0xff) != 0; i++){
        // .....
    }
    va_end(ap);
    release(&pr.lock);
    return 0;
}
```

第二个例子：

```
int printf(char *fmt, ...)
{
    va_list ap;
    int i, cx, c0, c1, c2;
    char *s;
    acquire(&pr.lock);
    va_start(ap, fmt);
    for(i = 0; (cx = fmt[i] & 0xff) != 0; i++){
        // .....
    }
    va_end(ap);
    release(&pr.lock);
    return 0;
}
```

```
// lock to avoid interleaving concurrent printf's.
static struct {
    struct spinlock lock;
} pr;
```

第二个例子：

```
int printf(char *fmt, ...)
{
    va_list ap;
    int i, cx, c0, c1, c2;
    char *s;
    acquire(&pr.lock);
    va_start(ap, fmt);
    for(i = 0; (cx = fmt[i] & 0xff) != 0; i++){
        // .....
    }
    va_end(ap);
    release(&pr.lock);
    return 0;
}
```

```
// lock to avoid interleaving concurrent printf's.
static struct {
    struct spinlock lock;
} pr;
```

```
void main()
{
    uartinit();
    printfinit();
    // .....
    scheduler();
}
```

第二个例子：

```
int printf(char *fmt, ...)
{
    va_list ap;
    int i, cx, c0, c1, c2;
    char *s;
    acquire(&pr.lock);
    va_start(ap, fmt);
    for(i = 0; (cx = fmt[i] & 0xff) != 0; i++){
        // .....
    }
    va_end(ap);
    release(&pr.lock);
    return 0;
}
```

```
// lock to avoid interleaving concurrent printf's.
static struct {
    struct spinlock lock;
} pr;
```

```
void main()
{
    uartinit();
    printfinit();
    // .....
    scheduler();
}
```

```
void printfinit(void)
{
    initlock(&pr.lock, "pr");
}
```

{

```
int i, cx, c0,
```

```
acquire(&pr.lock)
```

```
for(i = 0; (c
```

.....

```
va_end(ap);
```

```
va_end(ap);
return 1;
}
```

```
release(&pr.10);
return 0;
```

```
return 0;
```

▶ [Learn more about our services](#)

```
static struct {
```

```
pr;
```

```
} pr;
```

Task 1: Created!

[illegible]

```
11111111111111111111111111111111>Task 0: Created!
```

[illegible][illegible]

```
000000000000000000000000000000000000000000000><111111
```

[illegible]

```
1111111111111111111><0000000000000000000000000000000000000000000
```

[illegible][illegible]

```
1111111111><0000000000000000000000000000000000000000000000
```

[illegible][illegible]

 α'');

第三个例子：

kernel/proc.c

```
int nextpid = 1;
```

```
int allocpid()
{
    int pid;

    pid = nextpid;
    nextpid = nextpid + 1;

    return pid;
}
```

第三个例子：

kernel/proc.c

```
int nextpid = 1;  
struct spinlock pid_lock;
```

```
int allocpid()  
{  
    int pid;  
  
    pid = nextpid;  
    nextpid = nextpid + 1;  
  
    return pid;  
}
```

第三个例子：

kernel/proc.c

```
int nextpid = 1;
struct spinlock pid_lock;
```

```
void procinit(void)
{
    // .....
    initlock(&pid_lock, "nextpid");
    // .....
}
```

```
void main()
{
    uartinit();
    // .....
    procinit(); // process table
    // .....
    scheduler();
}
```

```
int allocpid()
{
    int pid;

    pid = nextpid;
    nextpid = nextpid + 1;

    return pid;
}
```

第三个例子：

kernel/proc.c

```
int nextpid = 1;
struct spinlock pid_lock;
```

```
void procinit(void)
{
    // .....
    initlock(&pid_lock, "nextpid");
    // .....
}
```

```
void main()
{
    uartinit();
    // .....
    procinit(); // process table
    // .....
    scheduler();
}
```

```
int allocpid()
{
    int pid;

    acquire(&pid_lock);
    pid = nextpid;
    nextpid = nextpid + 1;
    release(&pid_lock);

    return pid;
}
```

死锁 (Deadlock) 问题

- 什么是死锁以及死锁为何会发生：

当控制流执行路径中会涉及多个锁，并且这些控制流执行路径获取 (acquire) 锁的顺序不同时就可能会发生死锁问题



A

acquire (A)

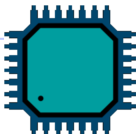


B

acquire (B)

acquire (B)

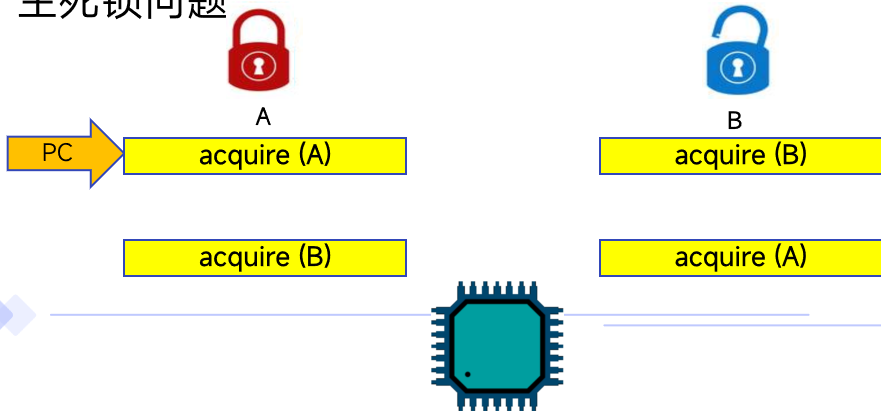
acquire (A)



死锁 (Deadlock) 问题

- 什么是死锁以及死锁为何会发生：

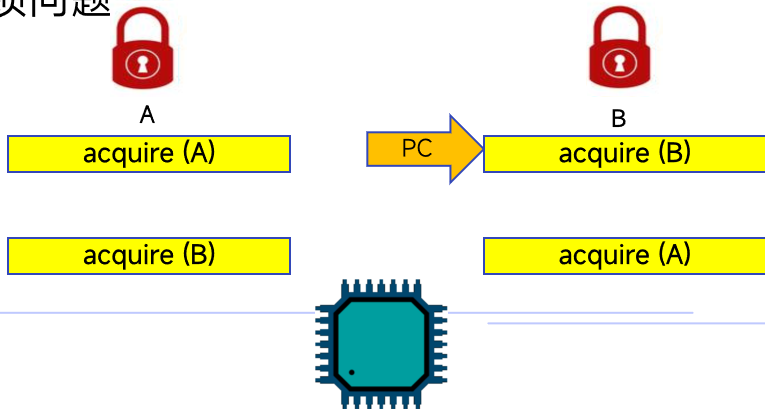
当控制流执行路径中会涉及多个锁，并且这些控制流执行路径获取 (acquire) 锁的顺序不同时就可能会发生死锁问题



死锁 (Deadlock) 问题

- 什么是死锁以及死锁为何会发生：

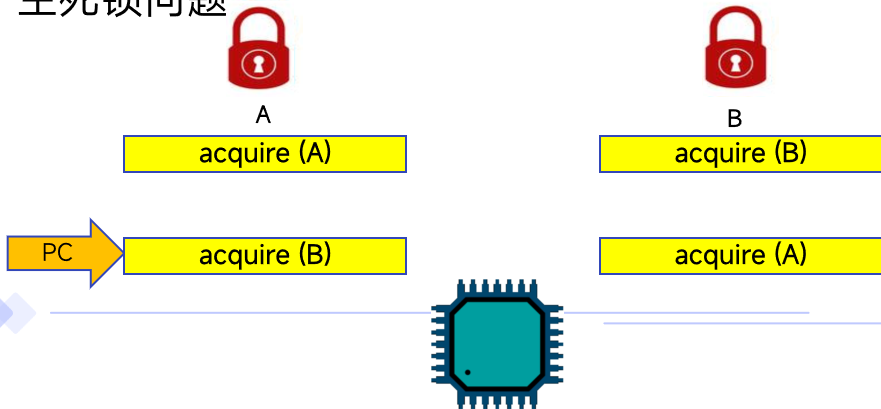
当控制流执行路径中会涉及多个锁，并且这些控制流执行路径获取 (acquire) 锁的顺序不同时就可能会发生死锁问题



死锁 (Deadlock) 问题

- 什么是死锁以及死锁为何会发生：

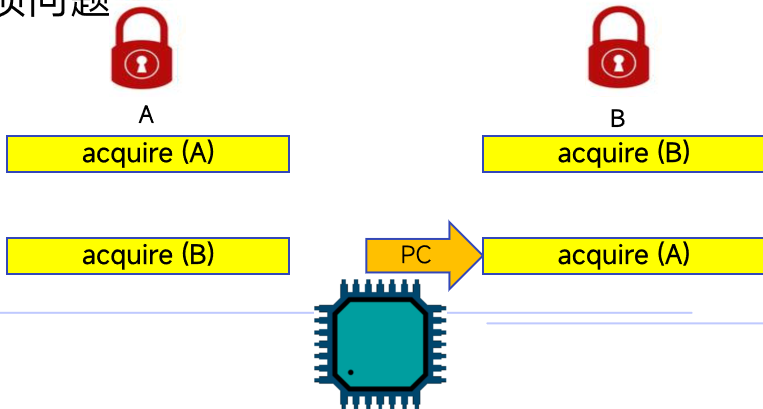
当控制流执行路径中会涉及多个锁，并且这些控制流执行路径获取 (acquire) 锁的顺序不同时就可能会发生死锁问题



死锁 (Deadlock) 问题

- 什么是死锁以及死锁为何会发生：

当控制流执行路径中会涉及多个锁，并且这些控制流执行路径获取 (acquire) 锁的顺序不同时就可能会发生死锁问题



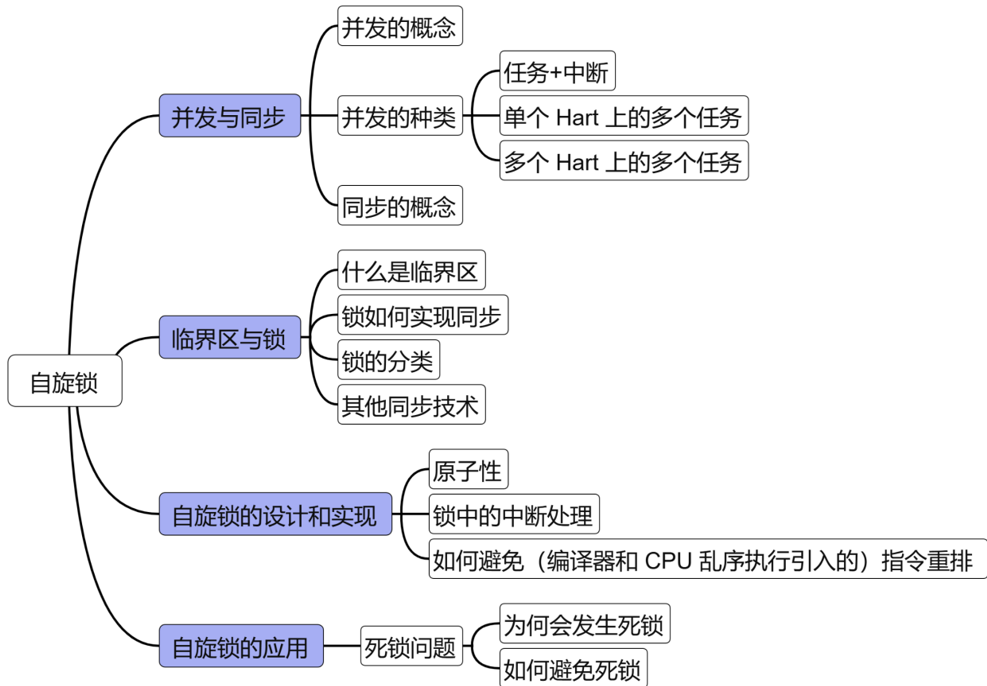
死锁 (Deadlock) 问题

- 如何解决死锁:

- ✓ 调整获取 (acquire) 锁的顺序, 按照固定顺序加锁。这是最常用且有效的方法。如果所有线程都严格按照相同的顺序去获取锁, 就不会形成循环等待。
- ✓ 尽可能防止任务在持有一把锁的同时申请其它的锁。
- ✓ 尽可能少用锁, 尽可能少并发。



本章总结



谢谢

